

UE – Ordonnancement et développement d'application temps réel

Initiation au langage de description des STR

Cours d'introduction à SDL :

<https://www.youtube.com/watch?v=0MG4l1Hqlb4>

Outil utilisé: **Pragmastudio**

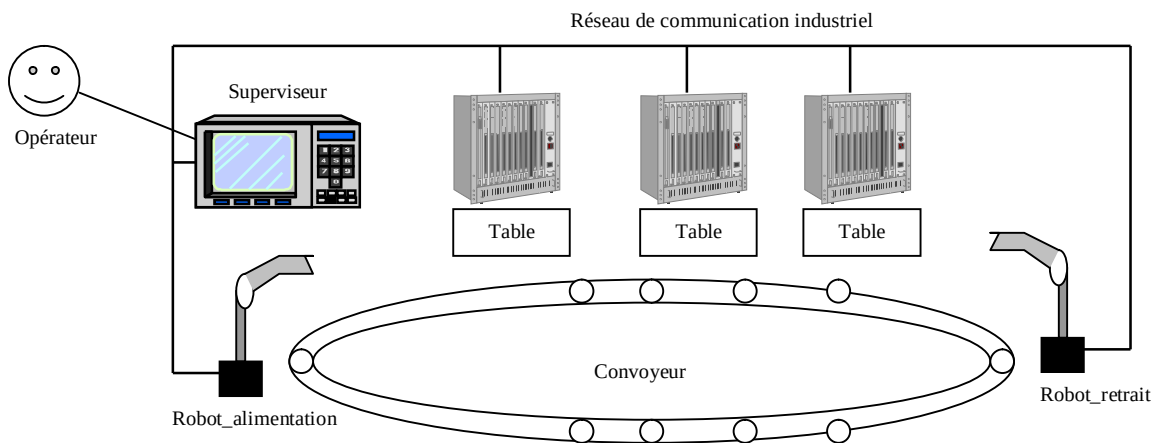
Premier pas avec SDL de Programdev : **suivre pas à pas** l'exemple du tutorial

<http://www.pragmadev.com/downloads/Manuals/Tutorial.pdf>

https://www.pragmadev.com/downloads/index_fr.html

STRE - Etude de cas

On considère un atelier de fabrication « fictif » composé d'un convoyeur, d'un robot d'alimentation de pièces à usiner, de plusieurs machines d'usinage et d'un robot de retrait de pièces usinées. L'ensemble est relié par un réseau de communication (voir figure ci-dessous).



Le superviseur reçoit des ordres de son environnement représenté ici par un opérateur. On suppose que le convoyeur tourne en permanence et que sa vitesse est choisie de manière adéquate pour que les robots et les machines puissent manipuler les pièces sans problème.

On suppose que chaque pièce est accompagnée d'un code qui spécifie l'opération d'usinage à lui appliquer et que chaque machine est spécialisée dans un processus de fabrication spécifique (ex. percer, souder, tordre, etc.). Quand l'opérateur signale au superviseur l'arrivée d'une nouvelle pièce à usiner, le superviseur détermine la

machine qui va l'usiner et envoie un ordre au robot d'alimentation pour déposer la pièce sur le convoyeur et à la machine d'usinage concernée pour se préparer.

Lorsqu'une machine d'usinage retire une pièce qu'elle doit usiner, elle la dépose sur sa table et envoie un signal de libération du convoyeur au superviseur. Chaque machine d'usinage possède sa propre table d'usinage sur laquelle elle dépose la pièce durant l'opération d'usinage. Lorsqu'une machine d'usinage termine son travail, elle envoie un compte rendu au superviseur et attend de celui-ci un ordre pour déposer la pièce sur le convoyeur. Lorsqu'un compte rendu de fin d'usinage est reçu, le superviseur attend que le convoyeur soit libre, ensuite il envoie un ordre à la machine d'usinage concernée pour déposer la pièce sur le convoyeur et un ordre au robot de retrait pour retirer la pièce afin de l'envoyer au dépôt. Quand la pièce est retirée un compte rendu est renvoyé au superviseur par le robot de retrait.

On suppose que tous les équipements (robots et machines d'usinage) sont munis de capteurs pour détecter l'arrivée de pièce et leur départ du convoyeur et que les robots et machines peuvent manipuler les pièces alors que le convoyeur est en mouvement.

Le superviseur joue le rôle de chef d'orchestre de la cellule de fabrication. L'arrivée des pièces est asynchrone. Ainsi, le superviseur peut avoir plusieurs pièces en cours d'usinage et plusieurs pièces en attente d'usinage.

Partie I : conception d'application temps réel

Etape 1 : Spécification en SDL

Faire une spécification SDL des sous-systèmes précédents (superviseur, robots et machines d'usinage) en SDL. On s'attachera à définir les diagrammes de séquence utiles et les diagrammes d'états-transitions pour décrire le comportement des sous-systèmes (montrant notamment le parallélisme des activités et leur synchronisation). L'opérateur et les capteurs constituent l'environnement du système à spécifier en SDL. Aussi, le réseau de communication sera décrit par un bloc dont la spécification n'est pas demandée.

Bien réfléchir à la conception des activités parallèles qui s'exécutent sur le superviseur avant de commencer leur spécification en SDL.

Etape 2 : Introduction de contraintes de temps réel pour la gestion des anomalies

On revisite le système décrit précédemment pour rajouter les contraintes suivantes :

- Le robot d'alimentation effectue l'opération de placement de pièces sur le convoyeur au bout de 30 secondes au maximum, s'il n'y arrive pas, il génère un signal d'anomalie vers le superviseur qui fait passer le système dans un état de défaillance.
- Si une machine d'usinage n'a pas retiré la pièce au bout de 20 secondes, le système passe dans un état de défaillance.
- Si une machine n'a pas fini son travail au bout de 5 minutes, le superviseur avertit l'opérateur en déclarant la machine en panne, mais n'arrête pas le système.
- Si le robot de retrait n'a pas fini son travail au bout de 30 secondes, le superviseur fait passer le système dans un état de défaillance.

Revoir la spécification fournie à l'étape 1 pour tenir compte des besoins ci-dessus.

Etape 3 (facultative) : Programmation du superviseur de l'étape 1 à l'aide des threads Python.

Partie II : Ordonnancement temps réel et analyse d'ordonnançabilité

Avant de mettre en place la cellule de fabrication décrite précédemment, on développe un prototype logiciel **de simulation** qui permet d'analyser les aspects temps réel de la cellule de fabrication. Pour cela, on développe un simulateur avec les considérations suivantes :

- Le simulateur fonctionne sur un seul processeur (donc on fera de l'ordonnancement mono processeur). Il n'y a plus de réseau de communication et les échanges éventuels se feront via la mémoire commune.
- On considère que le système de fabrication est composé de plusieurs ressources partagées ; chaque ressource correspond à une machine d'usinage et le temps d'utilisation d'une ressource correspond au temps d'usinage sur la machine correspondante.
- On considère, comme c'est souvent le cas dans les systèmes de production automatisés, que les machines d'usinage ne sont pas préemptibles (c.-à-d. que l'on n'arrête pas une machine qui en train d'usiner une pièce pour lui mettre une autre pièce à la place, puis reprendre la pièce précédente, etc.). Donc l'accès aux ressources partagées (les machines d'usinage) se fait en section critique.
- On suppose qu'il n'y a pas d'anomalie de fonctionnement des machines ou du convoyeur.

On va étudier plusieurs modes de fabrication (ou plus exactement de simulation de la fabrication) pour comprendre et utiliser les algorithmes d'ordonnancement de tâches temps réel.

Etape 1 : Etude d'algorithmes d'ordonnancement de tâches temps réel

On s'intéresse à plusieurs modes de fabrication pour illustrer l'utilisation des principaux algorithmes d'ordonnancement.

1) Mode 1 (tâches périodiques indépendantes)

Dans un premier temps, on considère qu'il n'y a qu'une seule machine d'usinage et qu'il y a quatre types de pièces à usiner. Chaque type de pièce est produit avec une période donnée et prend un certain temps d'usinage connu à l'avance. Ainsi, dans le simulateur, chaque type de pièce est modélisé par une tâche périodique dont la période et la durée d'exécution sont connues. La période de la tâche correspond à la période d'usinage du type de pièce et le WCET de la tâche correspond à la durée maximale d'usinage. Les pièces physiques sont des instances des types de pièces ; chaque exécution d'une tâche périodique est une instance de la tâche logicielle correspondant à un type de pièces. Au démarrage du simulateur, toutes les tâches périodiques sont prêtes pour s'exécuter. Dans le monde réel, ce sont les types de pièces qui sont prêts pour être fabriqués périodiquement.

- Utiliser RM pour faire l'ordonnancement des tâches périodiques. Donner un cas d'utilisation où l'ordonnancement n'est pas faisable (car la demande des tâches dépasse les capacités d'un processeur)
- Même question avec EDF.

Valeurs numériques à tester :

- Exemple 1 = { $T_1(P_1 = 8, C_1 = 2)$, $T_2(P_2 = 10, C_2 = 2)$, $T_3(P_3 = 8, C_3 = 1)$, $T_4(P_4 = 16, C_4 = 2)$ }
- Exemple 2 = { $T_1(P_1 = 8, C_1 = 3)$, $T_2(P_2 = 16, C_2 = 3)$, $T_3(P_3 = 4, C_3 = 1)$, $T_4(P_4 = 8, C_4 = 1)$ }
- Exemple 3 = { $T_1(P_1 = 6, C_1 = 2)$, $T_2(P_2 = 9, C_2 = 3)$, $T_3(P_3 = 15, C_3 = 3)$, $T_4(P_4 = 8, C_4 = 2)$ }
- Exemple 4 = { $T_1(P_1 = 6, C_1 = 2)$, $T_2(P_2 = 12, C_2 = 4)$, $T_3(P_3 = 24, C_3 = 4)$, $T_4(P_4 = 6, C_4 = 1)$ }

2) Mode 2 (tâches périodiques et tâches apériodiques indépendantes)

En plus des quatre types de pièces fabriquées périodiquement, des pièces à fabrication apériodique sont soumises par l'opérateur au système réel. Une pièce à usinage apériodique est décrite par son temps d'usinage et éventuellement le temps maximum pour finir l'usinage. Dans le simulateur, cela revient à associer une tâche apériodique à chaque pièce fabriquée de manière apériodique. Une tâche apériodique est définie par son instant d'arrivée, sa durée d'exécution et éventuellement son échéance. On utilise des **serveurs** pour gérer des (c.-à-d. allouer le processeur aux) tâches apériodiques. Il y a deux principales façons de fixer les paramètres d'un serveur de tâches apériodiques :

- de manière aléatoire : c'est simple à mettre en œuvre, mais c'est une solution qui ne tient pas compte de la réalité d'apparition des phénomènes aléatoires dans les STRE.

- de manière statistique : il s'agit de faire une étude des instants (aléatoires) d'apparition des tâches apériodiques et des durées d'exécution demandées et ensuite dériver un modèle statistique. Le plus simple des modèles statistiques consiste à utiliser la moyenne des intervalles inter-arrivée des tâches apériodiques pour fixer la période du serveur et la moyenne des durées d'exécution pour fixer la capacité du serveur.

Dans les deux cas, il y a une condition à respecter : il ne faut pas que le taux d'utilisation du processeur par le serveur ne dépasse pas ce qui est laissé libre par les tâches périodes. En d'autres termes, la somme des taux d'utilisation de toutes les tâches ne peut pas dépasser 100% du temps processeur, sinon il y a des fautes temporelles.

On ne s'occupe pas des échéances des tâches apériodiques et on vous demande d'étudier l'ordonnancement des tâches apériodiques en utilisant :

- a) un serveur à scrutation
- b) un serveur sporadique
- c) la technique *Slack-stealing* avec EDF.

Valeurs numériques à tester :

- I. Tâches périodiques = { $T_1(P_1 = 8, C_1 = 2)$, $T_2(P_2 = 16, C_2 = 2)$, $T_3(P_3 = 8, C_3 = 1)$, $T_4(P_4 = 8, C_4 = 1)$ }
- II. Période du serveur = 16, Capacité du serveur = 4
- III. Période du serveur = 6, Capacité du serveur = 2
- IV. Tâches apériodiques Tap_1 (instant d'arrivée = 3, $C_{Tap1} = 2$), Tap_2 (instant d'arrivée = 4, $C_{Tap2} = 5$)

3) Mode 3 (tâches avec des contraintes de précédence)

Dans une cellule de fabrication, un séquençement de fabrication de pièce est exigé quand les pièces fabriquées par la cellule vont être assemblées dans un certain ordre dans une cellule suivante (par exemple, une cellule de montage). Dans ce mode, on considère deux groupes composé chacun de quatre types de pièces produites périodiquement. Toutes les pièces d'un même groupe sont produites avec la même période et un ordre précis est imposé (**contraintes de précédence**). On note $TP_{i,j}$ la tâche correspondant au type i de pièces du groupe j . L'ordre de précédence à respecter est : $TP_{1,j} \rightarrow TP_{2,j} \rightarrow TP_{3,j} \rightarrow TP_{4,j}$ ($j = 1, 2$ et $X \rightarrow Y$ signifie que X précède Y)

- a) Appliquer les règles de transformation de tâches ordonnancées par RM pour prendre en compte les contraintes de précédence.
- b) Même question avec EDF.

Valeurs numériques à tester :

- I. Périodes (P), dates de réveil (r) et durée d'exécution (C) des tâches :
{ $T_{1,1}(P = 20, r = 1, C = 2)$, $T_{2,1}(P = 20, r = 0, C = 3)$, $T_{3,1}(P = 20, r = 2, C = 4)$, $T_{4,1}(P = 20, r = 0, C = 2)$ }
{ $T_{1,2}(P = 30, r = 0, C = 4)$, $T_{2,2}(P = 30, r = 2, C = 3)$, $T_{3,2}(P = 30, r = 0, C = 4)$, $T_{4,2}(P = 30, r = 3, C = 4)$ }

4) Mode 4 (tâches avec contraintes de partage de ressources)

On suppose que l'on a uniquement quatre types de pièces à fabrication périodique avec des contraintes de partage de ressources. Dans ce mode, on considère que les types de pièces font appel à plusieurs outils d'usinage (perçage, soudure, peinture...).

Chaque type de pièce est décrit par : sa période de fabrication, une liste ordonnée d'utilisation de ressources et la durée fabrication sur chaque pièce. Une pièce peut nécessiter une, deux ou plusieurs ressources de fabrication simultanément.

Dans le simulateur, chaque type de pièce est modélisé par une tâche périodique dont le corps est composé de demande et de libération de ressources partagées (qui simulent les outils de fabrication). Une tâche est décrite par sa période, la durée d'exécution dans chaque section critique.

On vous demande de :

- appliquer le protocole PIP (*priority inheritance protocol*) pour gérer l'accès aux ressources partagées ; Observer les situations d'interblocage avec PIP.
- appliquer le protocole PCP (*priority ceiling protocol*) pour gérer l'accès aux ressources partagées.

Valeurs numériques à tester :

$TP_1 : r = 1 ; P = 20 , LR = (r_1 [1] ; \{ r_1 [1] \parallel r_2 [1] \} ; \{ r_3 [1] \parallel r_4 [1] \})$

$TP_2 : r = 2 ; P = 30 , LR = (\{ r_4 [2] \parallel r_1 [2] \} ; r_1 [2])$

$TP_3 : r = 1 ; P = 40 , LR = (r_3 [3] ; \{ r_4 [2] \parallel r_2 [3] \} ; \{ r_3 [1] \parallel r_4 [2] \})$

$TP_4 : r = 0 ; P = 50 , LR = (r_2 [1] ; \{ r_1 [2] \parallel r_2 [2] \parallel r_3 [2] \} ; \{ r_3 [1] \parallel r_4 [2] \})$

La notation ci-dessus se lit : pour le type de pièce i , on a la date de réveil (r) de la tâche correspondante, la période (P), la liste (LR) d'utilisation des ressources avec le temps d'occupation (spécifié par $[n]$) de la ressource. ';' signifie que les ressources sont utilisées en séquence et ' $\parallel$ ' signifie que les ressources sont utilisées en même temps (sections critiques imbriquées).

Remarque : PIP et PCP s'utilisent conjointement avec RM, EDF ou autre. Dans cet exemple, vous utiliserez RM.

Etape 2 : Analyse d'ordonnançabilité non-assistée

Reprendre les modes décrits précédemment et traiter manuellement le test d'ordonnançabilité des modèles de tâches proposés. On utilise les conditions d'ordonnançabilité des algorithmes d'ordonnancement testés précédemment (RM, EDF...)

Etape 3 : Analyse d'ordonnançabilité assistée

Parmi les rares outils d'analyse d'ordonnançabilité gratuits qui existent, il y a MAST et Cheddar. On vous demande de tester un des deux outils en reprenant les divers modes de gestion de la cellule de fabrication décrits précédemment :

- 1) **MAST** (*Modeling and Analysis Suite for real-Time applications*)

<https://mast.unican.es/>

- 2) **Cheddar**

<http://beru.univ-brest.fr/cheddar/>